Table of Contents

Icons **1**

Gestures **4**

Drag Gestures 4
Click Gesture 13
Handling Action Down and Action Up Events 14

Alerts **16**

Dialogs **19**

Exercise 9 (4 marks)Dialogs **21**

Tab Bars **24**

Top Tab Bar 24
Bottom Tab Bar 28

AppBar **28**

Icons

The icons available through the default Kotlin build are very limited. However, many more icons which can be used in your Kotline project are available at https://fonts.google.com/icons
Loading the extended icon library is not recommended.

Do not add this dependency because it will bundle all icons into your app whether you use them or not. Only use this dependency if your project will use many icons.

```
    implementation("androidx.compose.material:material-icons-extended:1.7.6")
```

You may use a sub selection of the extended icon library with the import statement:

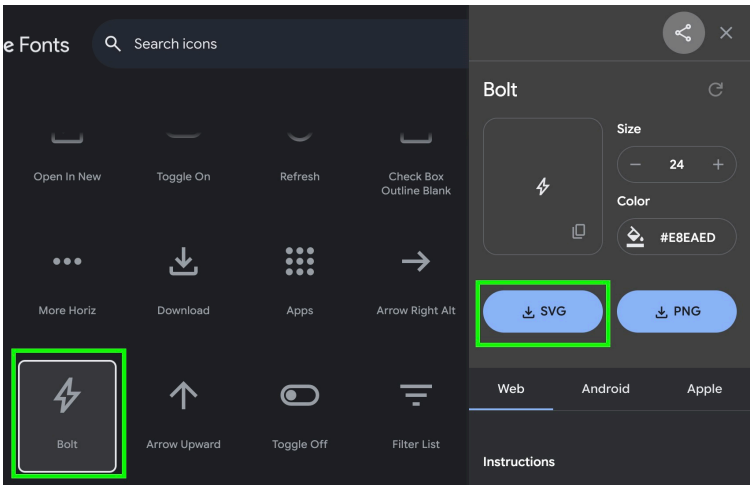
```
implementation("androidx.compose.material:material-icons-core:1.7.6")
```

Unfortunately, the non-extended library has a much smaller range of icons and the lack of variety is not satisfying. I personally find the variety to be inadequate for an app.

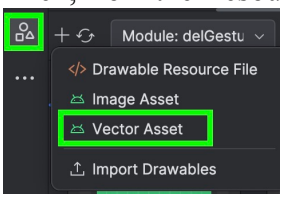
To keep you bundle size smaller and to only load the icons from https://fonts.google.com/icons that you need into your project, select the one you would like at a time and then download it as an SVG.

Example 1: Adding Icons

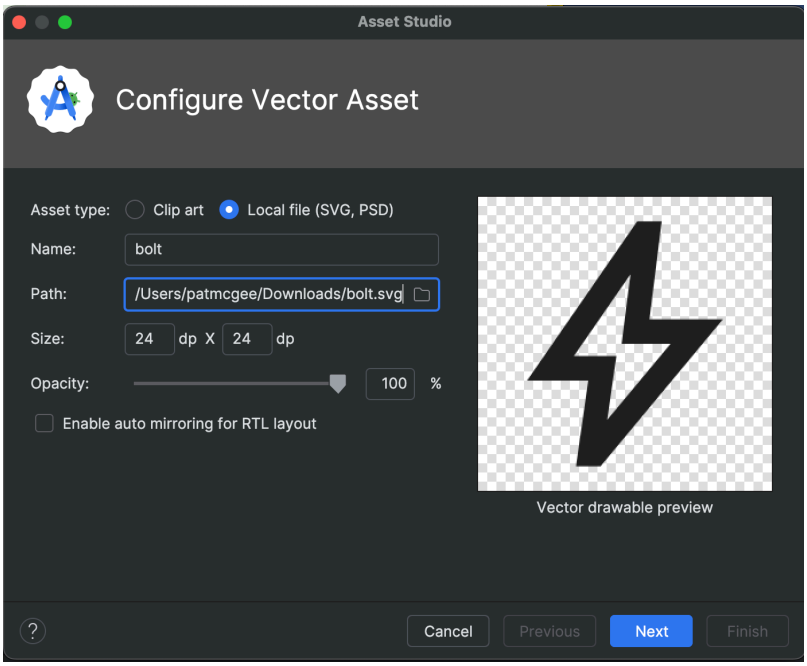
This example demonstrates how to add icons that are downloaded from the google icons site:



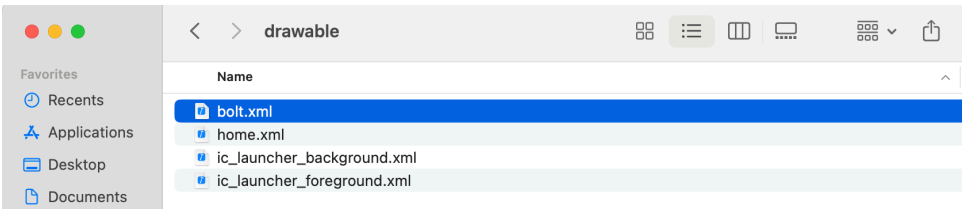
Then, from the Resource Manager, click + and choose Vector Asset.



After, navigate to the svg file that you downloaded and select it. When prompted select the vector asset and click Next.



Finishing these steps will convert the SVG to an XML file which will be stored in the drawables folder.

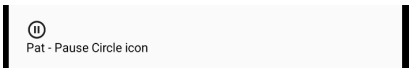


Now the icon can be loaded in your app.

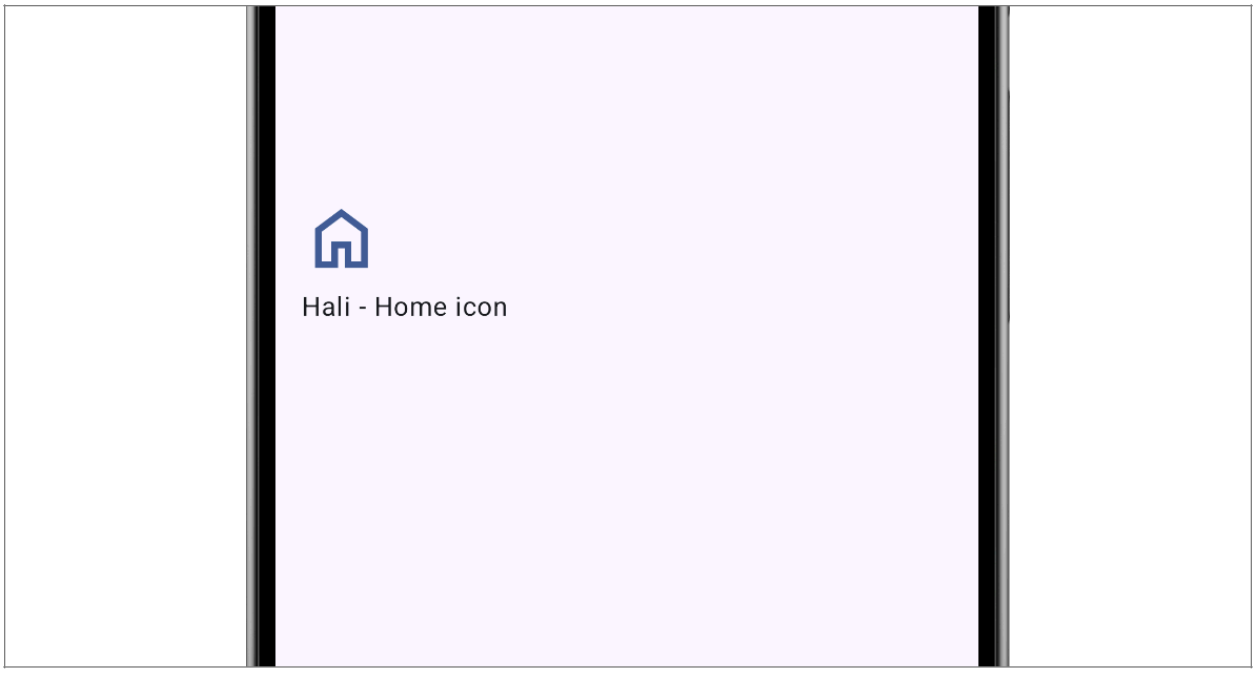
```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Column(modifier = Modifier.padding(20.dp)) {
                Spacer(modifier = Modifier.height(60.dp))
                Image(
                    painter = painterResource(id = R.drawable.bolt), // Custom icon
                    contentDescription = "Bolt Icon",
                    modifier = Modifier
                )
            }
        }
    }
}
```

Exercise 1 (1 mark)

Create an app with an icon from https://fonts.google.com/icons that starts with the first letter of your name. Show a screenshot of the icon running. Use a composable Text object to show your name and show the name of the icon. It should look something like this:



Show a screenshot of your icon here:



Gestures

In Jetpack Compose, the ***pointerInput*** property of a ***Modifier*** object can be used to detect and handle touch gestures like taps, drags, swipes, and more. *pointerInput* can detect touch events on a composable object in addition to tracking finger position as it moves across it.

Drag Gestures

Example 2: Enabling Pointer Input Gestures

This example shows how to detect horizontal drag gestures. The ***dragAmount*** is detected in the ***onHorizontalDragGestures*** handler.

```
@Composable
fun SwipeGestureDemo() {
    // Track horizontal offset for swipe gesture
    var offsetX by remember { mutableStateOf(0f) }

    Column {
        Text(offsetX.toString())

        Box(
            modifier = Modifier
                .fillMaxSize() // take up all available space in the container
                .background(Color.LightGray)
                .pointerInput(Unit) {
                    detectHorizontalDragGestures(
                        onHorizontalDrag = { _, dragAmount ->
                            offsetX += dragAmount // Update offset amount
                        }
                    )
                }
        ) {
            // Swipable Box
            Box(
                modifier = Modifier
                    .background(Color.Blue)
                    .offset { IntOffset(offsetX.roundToInt(), 0) } // horiz offset
                    .fillMaxSize(fraction = 0.5f) // Make box smaller
            )
        }
    }
}
```

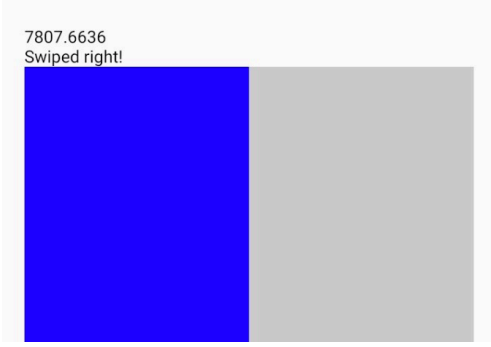
To launch this code, replace the contents of setContent {} with this version:

```
setContent {
    Column(modifier     = Modifier.padding(20.dp)) {
        Spacer(modifier = Modifier.height(60.dp))
        SwipeGestureDemo()
    }
}
```

Exercise 2 (3 marks)

Implement code to show a message which displays whether a person "Swiping right!" if the ***dragAmount*** is greater than 0. Show a message which displays if the person "Swiping left!" if the *dragAmount* is less than 0.

Note: Do not be concerned about what happens after the finger is lifted off the screen. This will be discussed soon.



Show your code here.

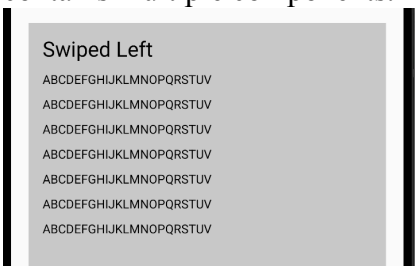
```
package com.example.lab

import androidx.compose.foundation.background
import androidx.compose.foundation.gestures.detectHorizontalDragGestures
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlin.math.roundToInt

@Composable
fun SwipeGestureDemo() {
    // Track horizontal offset for swipe gesture
    var offsetX by remember { mutableStateOf(0f) }
    var swipeMessage by remember { mutableStateOf("No swipe yet") }

    Column(
        modifier = Modifier.fillMaxSize()
```

```
    ) {
        // Display offset number at top
        Text(
            text = offsetX.toString(),
            fontSize = 20.sp,
            color = Color.Black,
            modifier = Modifier
                .padding(16.dp)

        )

        // Display swipe message
        Text(
            text = swipeMessage,
            fontSize = 20.sp,
            color = Color.Black,
            modifier = Modifier
                .padding(start = 16.dp, bottom = 8.dp)

        )

        Box(
            modifier = Modifier
                .fillMaxSize()
                .background(Color.LightGray)
                .pointerInput(Unit) {
                    detectHorizontalDragGestures(
                        onHorizontalDrag = { _, dragAmount ->
                            offsetX += dragAmount

                            // Exercise 2: Check swipe direction
```

```
                        if (dragAmount > 0) {
                            swipeMessage = "Swiping right!"
                        } else if (dragAmount < 0) {
                            swipeMessage = "Swiping left!"
                        }
                    }
                )
            }
        ) {
            // Swipable Box
            Box(
                modifier = Modifier
                    .background(Color.Blue)
                    .offset { IntOffset(offsetX.roundToInt(), 0) }
                    .fillMaxSize(fraction = 0.5f)
            )
        }
    }
}
```

Example 3: Checking for the end of a Swipe

This example displays the final swipe direction after the swipe has completed. The example also goes a little further to show how a swipeable property can be applied to a component which contains multiple components.



The detectHorizontalDragGestures handler uses an ***onDragStart*** handler to set a *totalDragX* amount to 0. The totalDragX amount is updated in the *onHorizontalDrag* handler. The ***onDragEnd*** handler is called when the finger is lifted from the screen.

To make this example work, replace SwipeGestureDemo() in Example 2 with this code:

```
@Composable
fun SwipeGestureDemo() {
    var horizMsg by remember { mutableStateOf("") }
    var totalDragX by remember { mutableStateOf(0f) }
    val thresholdPx = 80f

    // Touch surface for the whole column; content itself doesn't move
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.LightGray)
            .pointerInput(Unit) {
                detectHorizontalDragGestures(
                    onDragStart = { totalDragX = 0f },
                    onHorizontalDrag = { _, dragAmount ->
                        totalDragX += dragAmount
                    },
                    onDragEnd = {
                        horizMsg = when {
                            abs(totalDragX) < thresholdPx -> "No swipe"
                            totalDragX > 0f -> "Swiped Right"
                            else -> "Swiped Left"
                        }
                        Log.d("***", "Column: $horizMsg")
                    },
                    onDragCancel = {
                        horizMsg = "Cancelled"
                        Log.d("***", "Column: $horizMsg")
                    }
                )
            }
    ) {
        Column(
            modifier = Modifier
                .align(Alignment.TopStart)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            // status message
            Text(horizMsg, fontSize = 24.sp)

            // your exact text lines (not moveable)
            Text("ABCDEFGHIJKLMNOPQRSTUV")
            Text("ABCDEFGHIJKLMNOPQRSTUV")
            Text("ABCDEFGHIJKLMNOPQRSTUV")
            Text("ABCDEFGHIJKLMNOPQRSTUV")
            Text("ABCDEFGHIJKLMNOPQRSTUV")
            Text("ABCDEFGHIJKLMNOPQRSTUV")
```

Exercise 3 (3 marks)

Modify the code for the swipeable box container so it detects up and down swipes. Change the text to show your name.

Show the code for your modified SwipeGestureDemo() function here:

```
package com.example.lab


import androidx.compose.foundation.background
import androidx.compose.foundation.gestures.detectVerticalDragGestures
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import android.util.Log
import kotlin.math.abs
@Composable
fun SwipeGestureDemo() {
    var vertMsg by remember { mutableStateOf("") }
    var totalDragY by remember { mutableStateOf(0f) }
    val thresholdPx = 80f
```

```
Box(
    modifier = Modifier
        .fillMaxSize()
        .background(Color.LightGray)
        .pointerInput(Unit) {
            detectVerticalDragGestures(
                onDragStart = { totalDragY = 0f },
                onVerticalDrag = { _, dragAmount ->
                    totalDragY += dragAmount
                },
                onDragEnd = {
                    vertMsg = when {
                        abs(totalDragY) < thresholdPx -> "No swipe"
                        totalDragY > 0f -> "Swiped Down"
                        else -> "Swiped Up"
                    }
                    Log.d("***", "Column: $vertMsg")
                },
                onDragCancel = {
                    vertMsg = "Cancelled"
                    Log.d("***", "Column: $vertMsg")
                }
            )
        }
) {
    Column(
        modifier = Modifier
            .align(Alignment.TopStart)
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        // status message
        Text(vertMsg, fontSize = 24.sp)
```

```
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
        Text("Hali Imanpanah", fontSize = 32.sp)
    }
  }
}
```

Show a screenshot of your output here:

Click Gesture

Example 4: Making a control or control block clickable

This example transforms Example 3 so the text is clickable.

To build this example, replace the containing box Example 3 with this version which implements a clickable handler.

```
Box(
    modifier = Modifier
        .fillMaxSize()
        .background(Color.LightGray)
        .clickable {
            horizMsg = "Clicked"
            Log.d("***", "Column: $horizMsg")
        }
)
```

Handling Action Down and Action Up Events

There are other ways to capture finger press events. You may want to use a different method to handle finger press events especially if you need to also detect when a finger is lifted off the screen after a click.

Example 5: Handling Action Down Events

This example converts Example 4 so it will track finger down events with *pointerInteropFilter*. To build this example, replace the Box container in Example 3 with this version.

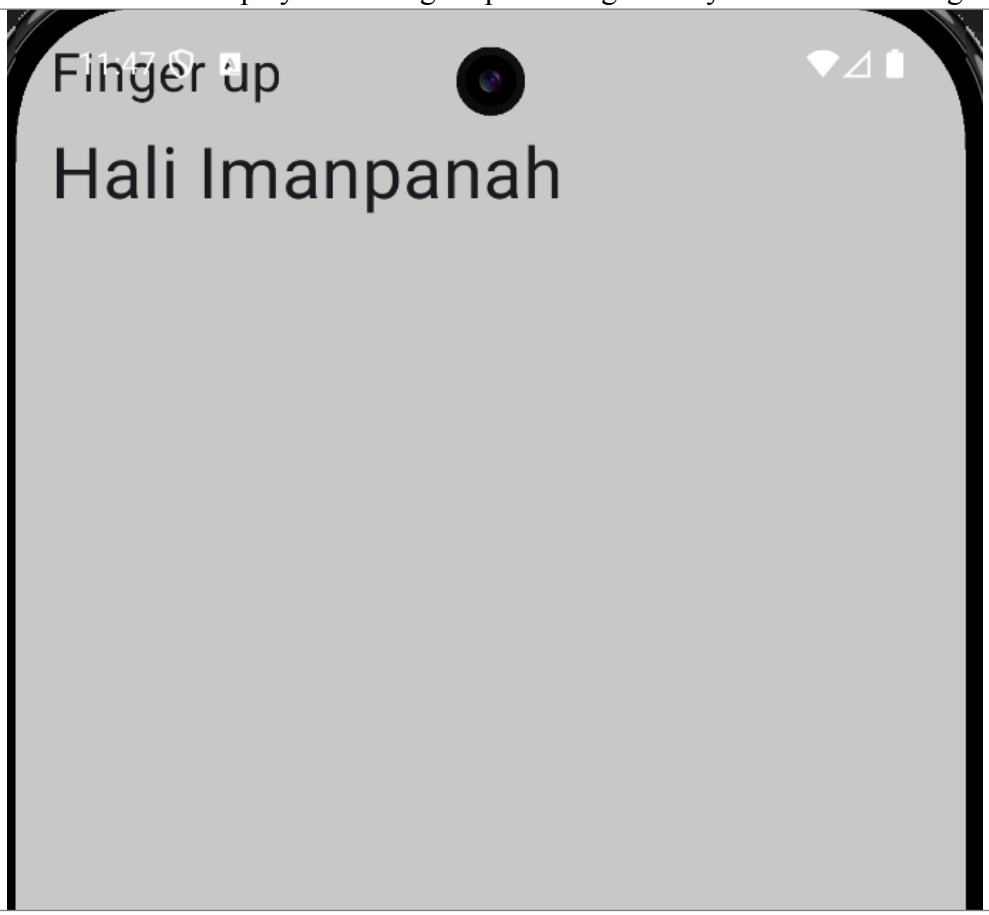
```
Box(
    modifier = Modifier
        .fillMaxSize()
        .background(Color.LightGray)
        // pointerInteropFilter gives you raw MotionEvent callbacks
        .pointerInteropFilter { event ->
            when (event.action) {
                MotionEvent.ACTION_DOWN -> {
                    horizMsg = "Finger down"
                    Log.d("***", "Column: $horizMsg")
                }
                // Code for handling ACTION_UP goes here.
            }
            true // consume the event
        }
)
```

Exercise 4 (2 marks)

Modify the code in Example 5 so it shows a “Finger up” message when ACTION_UP is detected in addition to displaying “Finger down” when ACTION_DOWN is detected. Show the modified event.action conditional block here:

```
.pointerInteropFilter { event ->
    when (event.action) {
        MotionEvent.ACTION_DOWN -> {
            vertMsg = "Finger down"
            Log.d("***", "ACTION_DOWN detected")
            true
        }
        else -> false
    }
}
```

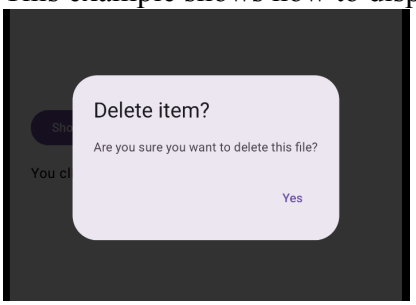
Show a screenshot which displays the “Finger up” message after you make the changes.



Alerts

Example 6: Alert Box

This example shows how to display a basic Alert Box.

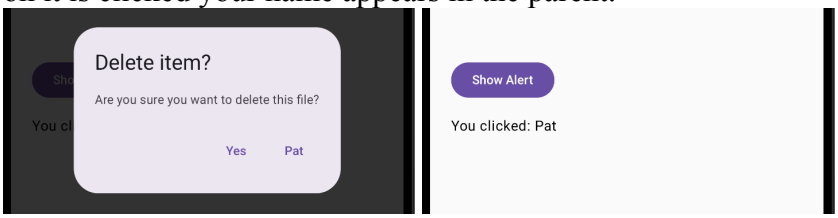


Here is the code:

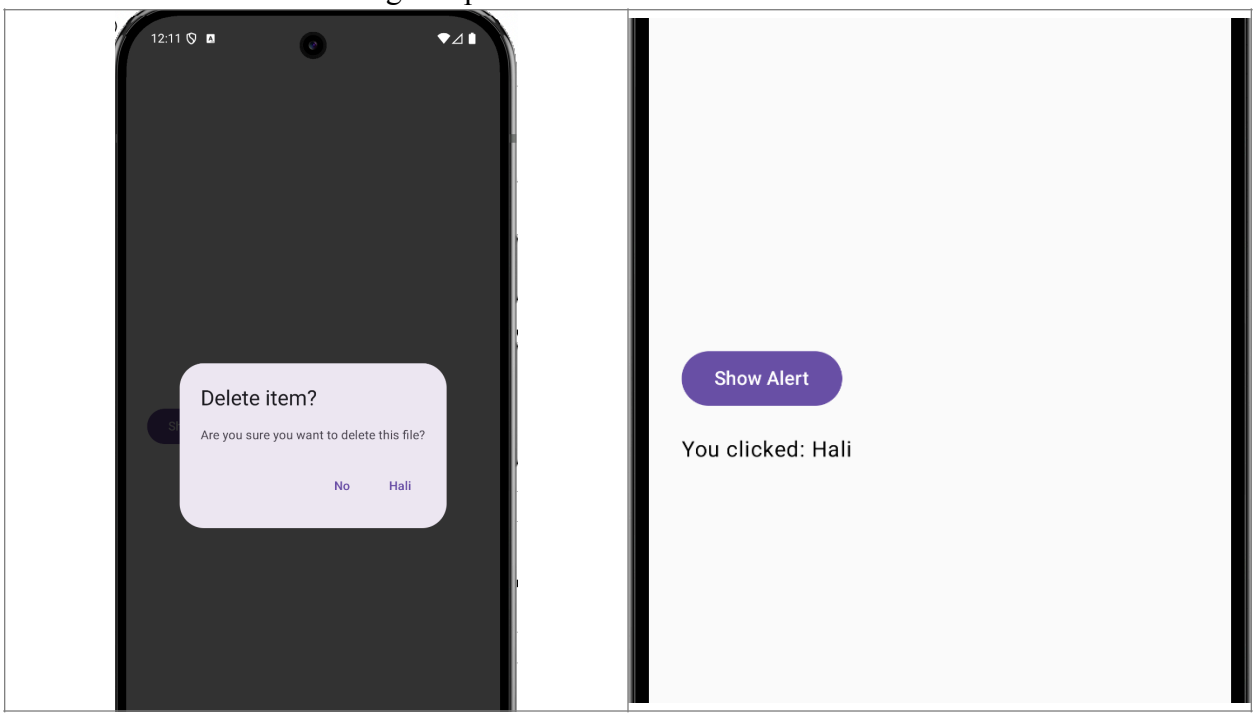
```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AlertDialogExample()
        }
    }
}

@Composable
fun AlertDialogExample() {
    var showDialog by remember { mutableStateOf(false) }
    var userChoice by remember { mutableStateOf("") }   // store Yes/No result

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Button(onClick = { showDialog = true }) {
            Text("Show Alert")
        }

        Spacer(modifier = Modifier.height(16.dp))

        if (userChoice.isNotEmpty()) {
            Text("You clicked: $userChoice",
            style = MaterialTheme.typography.bodyLarge)
        }
        if (showDialog) {
            ConfirmDeleteDialog(
                onConfirm = {
                    showDialog = false
                    userChoice = "Yes"
                }
            )
        }
    }
}

@Composable
fun ConfirmDeleteDialog(
    onConfirm: () -> Unit,
) {
    AlertDialog(
        onDismissRequest = onConfirm,
        title = { Text("Delete item?") },
```

Exercise 5 (2 marks)

Modify Example 6 to add another button with your name on it. When the button with your name on it is clicked your name appears in the parent.

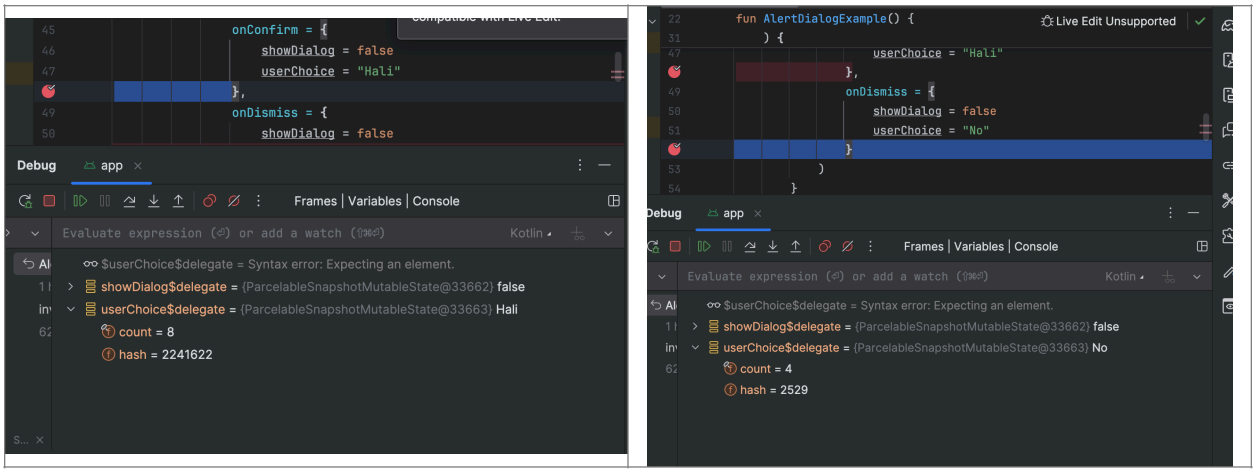


Show screenshots of the dialog and parent screen here:



Exercise 6 (2 marks)

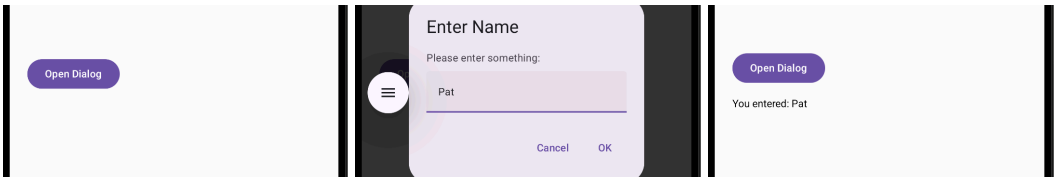
Set a breakpoint in the callback method within the parent for your new button created in Exercise 5. Show a screenshot of the active breakpoint while the program is halted in the parent callback after clicking the new button.



Dialogs

Example 7: Adding controls to Dialogs

This example shows a more advanced dialog which implements a TextField which passes the value received by the dialog back to the parent control.



Here is the code:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { AlertDialogExample() }
    }
}

@Composable
fun AlertDialogExample() {
    var showDialog by remember { mutableStateOf(false) }
    var userInput by remember { mutableStateOf("") } // holds result

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Button(onClick = { showDialog = true }) {
            Text("Open Dialog")
        }

        Spacer(Modifier.height(16.dp))

        if (userInput.isNotEmpty()) {
            Text("You entered: $userInput")
        }

        if (showDialog) {
            TextInputDialog(
                title = "Enter Name",
                onConfirm = { text ->
                    userInput = text
                    showDialog = false
                },
                onDismiss = { showDialog = false }
            )
        }
    }
}

@Composable
fun TextInputDialog(
    title: String,
    onConfirm: (String) -> Unit,
    onDismiss: () -> Unit
) {
    var text by remember { mutableStateOf("") }
```

Exercise 7 (1 mark)

What is the return type of the onDismiss? Why is that return type used?

`Unit` Because the callback performs an action but doesn't need to return a value back to the caller

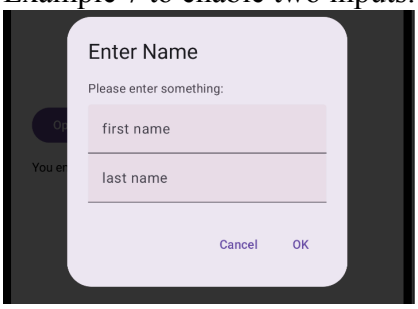
Exercise 8 (1 mark)

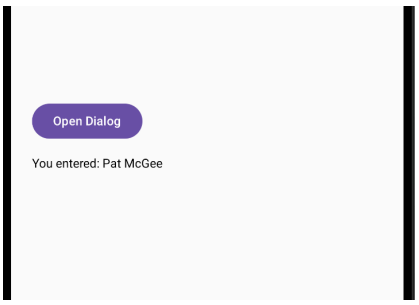
What is the purpose of the onDismissRequest handler in the alert dialog?

It ensures the dialog closes properly and updates the app state (like setting showDialog = false)

Exercise 9 (4 marks)Dialogs

Example 7 to enable two inputs.



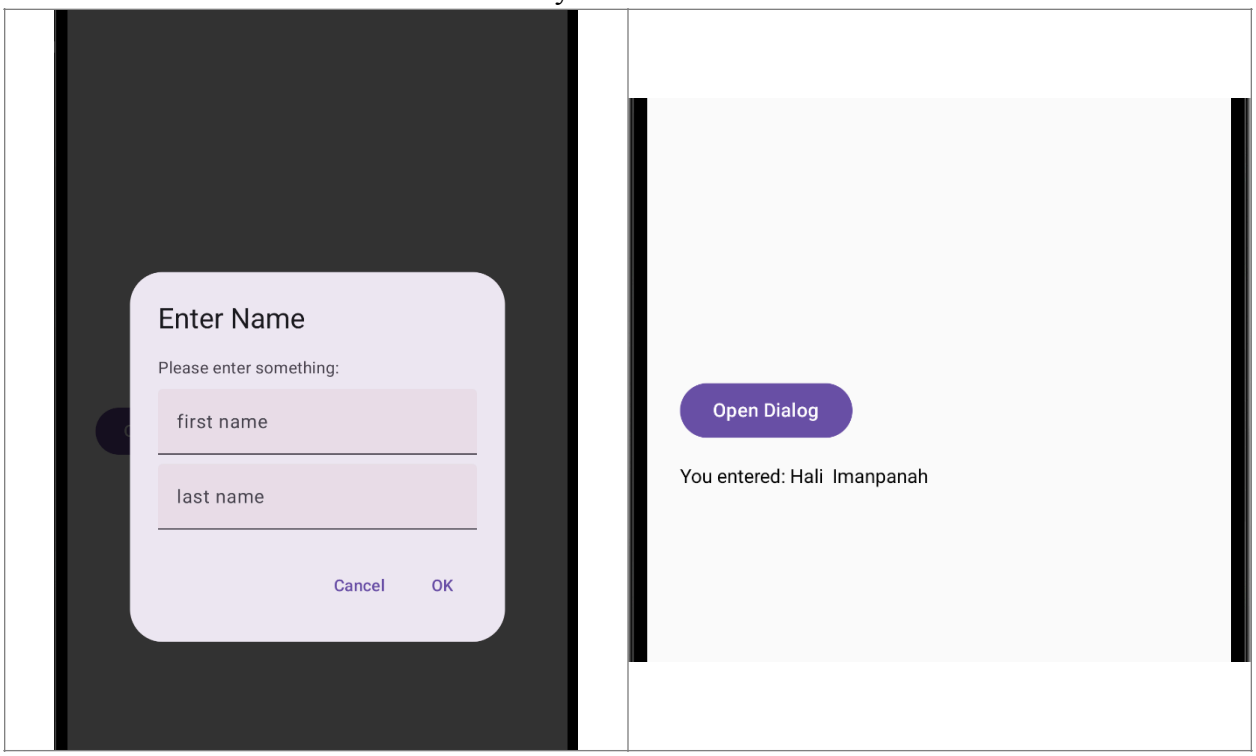


Show your revised TextInputDialog() function after making this change.

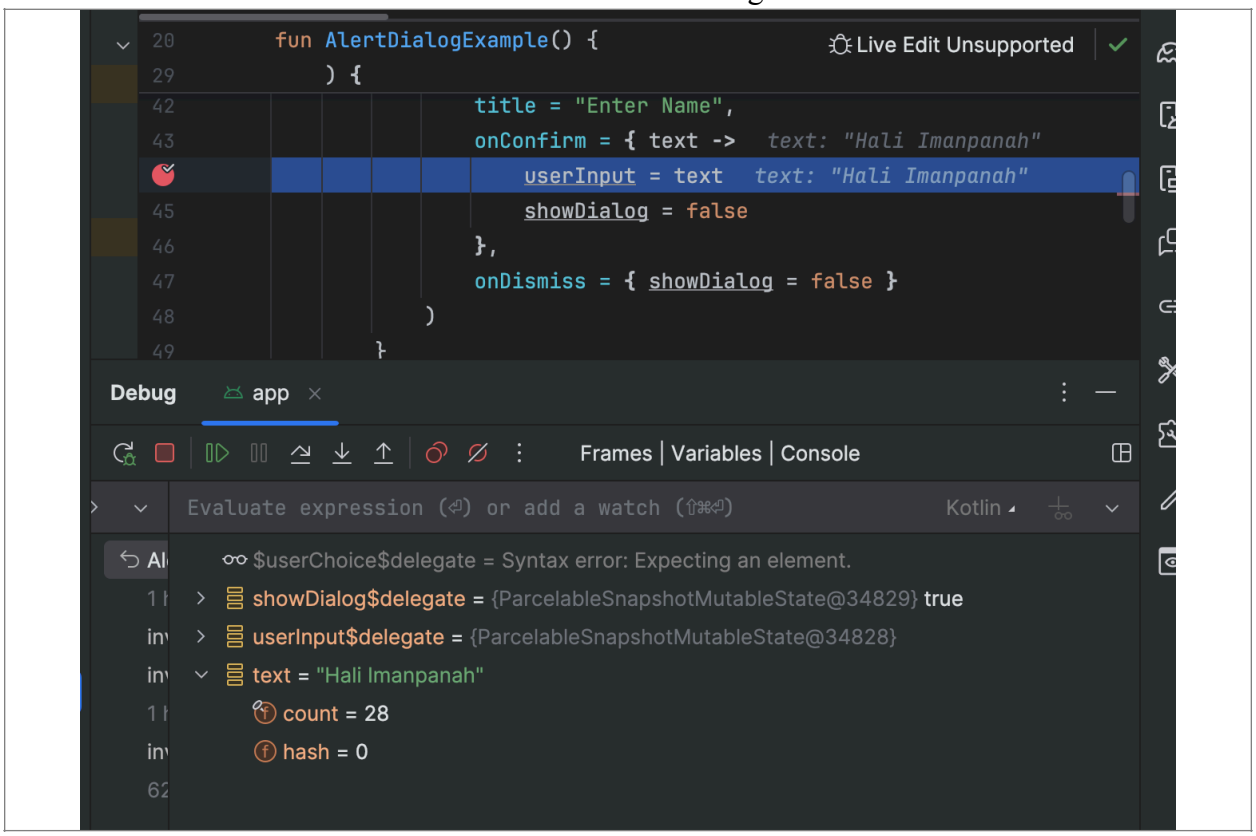
```
@Composable
fun TextInputDialog(
    title: String,
    onConfirm: (String) -> Unit,
    onDismiss: () -> Unit
) {
    var firstName by remember { mutableStateOf("") }
    var lastName by remember { mutableStateOf("") }


    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(title) },
```

```
        text = {
            Column {
                Text("Please enter something:")
                Spacer(Modifier.height(8.dp))
                TextField(
                    value = firstName,
                    onValueChange = { firstName = it },
                    placeholder = { Text("first name") },
                    singleLine = true
                )
                Spacer(Modifier.height(8.dp))
                TextField(
                    value = lastName,
                    onValueChange = { lastName = it },
                    placeholder = { Text("last name") },
                    singleLine = true
                )
            }
        },
        confirmButton = {
            TextButton(onClick = { onConfirm("$firstName $lastName") }) {
                Text("OK")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}
```

Show a screenshot of the text content with your name in it.



Show a screenshot of the program while it is halted at a breakpoint in the parent callback which receives the name information that is obtained in the dialog.

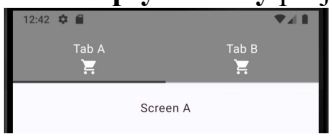


Tab Bars

Top Tab Bar

Example 8: Creating a Tab Bar Application

This app builds a tab-based application where each tab leads to a different screen. It starts with a new **Empty Activity** project.

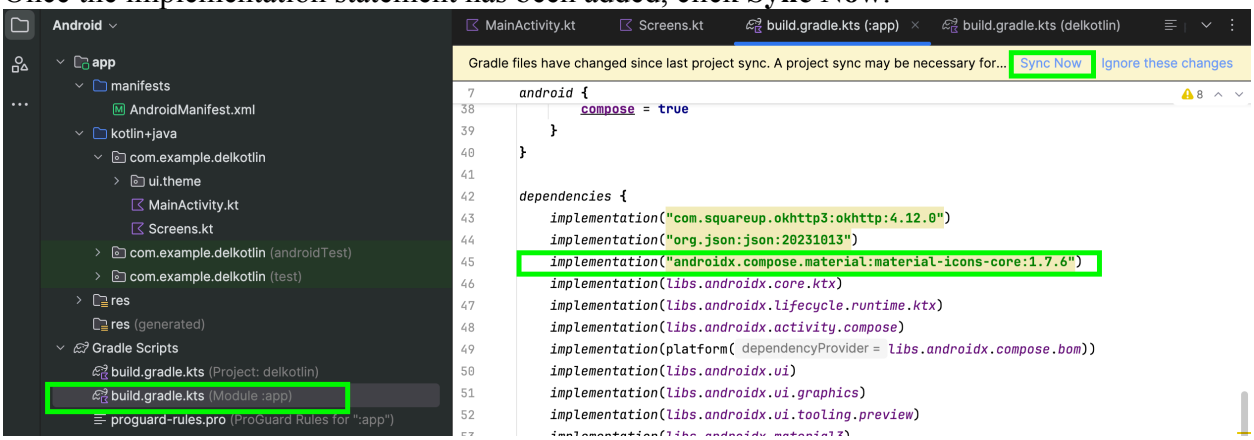


This code defines the body for each of the separate screens.

First, this example uses icons from the limited library for ease of use. (If you need a larger variety of icons, see Example 1.) To include the limited icons library, add this line of code to the dependences section in the **build.gradle.kts** file.

```
implementation("androidx.compose.material:material-icons-core:1.7.6")
```

Once the implementation statement has been added, click **Sync Now**.



Screens.kt

```
@Composable
fun ScreenA() {
    Column(
        Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center) {
        Text(text = "Screen A")
    }
}

@Composable
fun ScreenB() {
    Column(
        Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center) {
        Text(text = "Screen B")
    }
}
```

This code builds the TabRowScreens. InitializeTabsFromList() sets up a tab bar from a list of tabs. ShowTabBar() displays the tab bar. Inside MainActivity.kt add this:

```
@Composable
fun InitializeTabsFromList() {
    // List of tab titles
    val tabs = listOf("Tab A", "Tab B")

    ShowTabBar(
        tabs = tabs,
        contentScreens = listOf(
            { ScreenA() },
            { ScreenB() },
        ),
        modifier = Modifier.fillMaxSize(),
        containerColor = Color.Gray,
        contentColor = Color.White,
        indicatorColor = Color.DarkGray
    )
}

@Composable
fun ShowTabBar(
    tabs: List<String>,
    contentScreens: List<@Composable () -> Unit>,
    modifier: Modifier = Modifier,
    containerColor: Color,
    contentColor: Color,
    indicatorColor: Color
) {
    var selectedTabIndex by remember { mutableIntStateOf(0) }

    Scaffold(
        modifier = modifier,
        topBar = {
            TabRow(
                selectedTabIndex = selectedTabIndex,
                containerColor = containerColor,
                contentColor = contentColor,
                indicator = { tabPositions ->
                    SecondaryIndicator(
                        modifier =
                        Modifier.tabIndicatorOffset(tabPositions[selectedTabIndex]),
                        color = indicatorColor
                    )
                }
            ) {
                tabs.forEachIndexed { index, tabTitle ->
                    Tab(
                        selected = selectedTabIndex == index,
                        onClick = { selectedTabIndex = index }
```

Next replace the setContent() function in MainActivity with this version.

```
setContent {
    InitializeTabsFromList()
}
```

Exercise 10 (1 mark)

Why is remember used with mutableIntStateOf(0) inside ShowTabBar()?

Remember keeps the value saved when the screen refreshes. Without it, the selected tab would reset to 0 every time.

Exercise 11 (1 mark)

How does the TabRow know which tab is currently selected?

through the selectedTabIndex parameter, which holds the current tab's index number like 0 for first tab, 1 for second tab ,….

Exercise 12 (1 mark)

What does the indicator parameter do inside TabRow control?

draws the visual highlight like line/bar under the currently selected tab to show which tab is active.

Exercise 13 (1 mark)

What visual role does the SecondaryIndicator play in helping users understand tab state?

provides a visual like an underline or coloured bar) which shows users which tab is currently active/selected, making it clear where they are in the navigation.

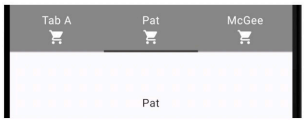
Exercise 14 (1 mark)

Why is the Box composable used at the bottom of the Scaffold?

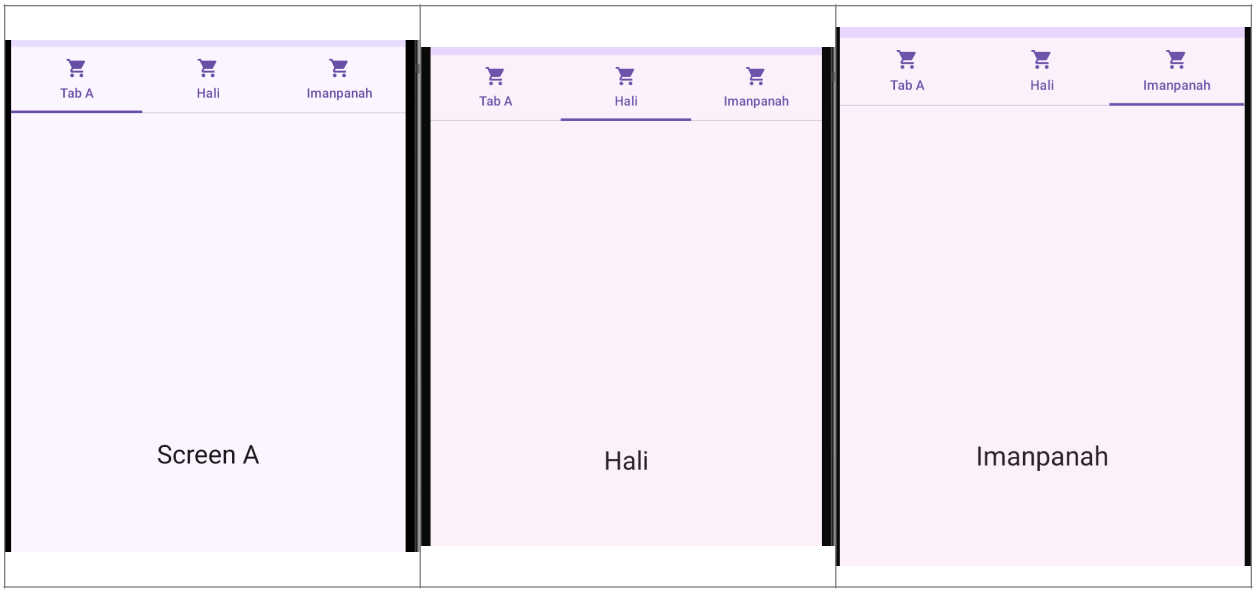
to hold and display the content for each tab. It acts like a container with showing different screens based on which tab is selected

Exercise 15 (4 marks)

Modify the code to show tabs for three screens. The last two tabs need to have your first and last name and the screens need to have your names in the center of the screen. (I shortened the images to save space so my name appears closer to the top than it really is.).



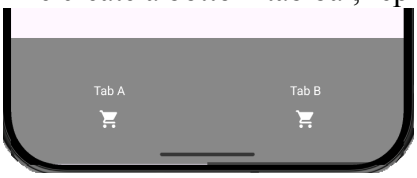
Show screenshots of your adjusted tabs and screens after adding the third screen.



Bottom Tab Bar

Example 9: Creating a Bottom Tab Bar

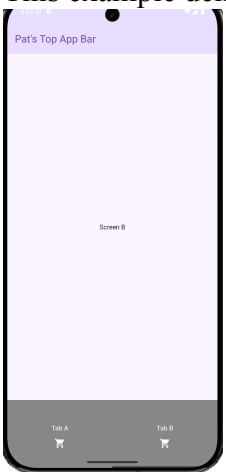
The create a bottom tab bar, replace "**topBar** =" in Example 8 with "**bottomBar** =".



AppBar

Example 10: Adding an App Bar

This example demonstrates how to add an AppBar to the solution from Example 9.



In a separate file named TopAppBar, add this code:

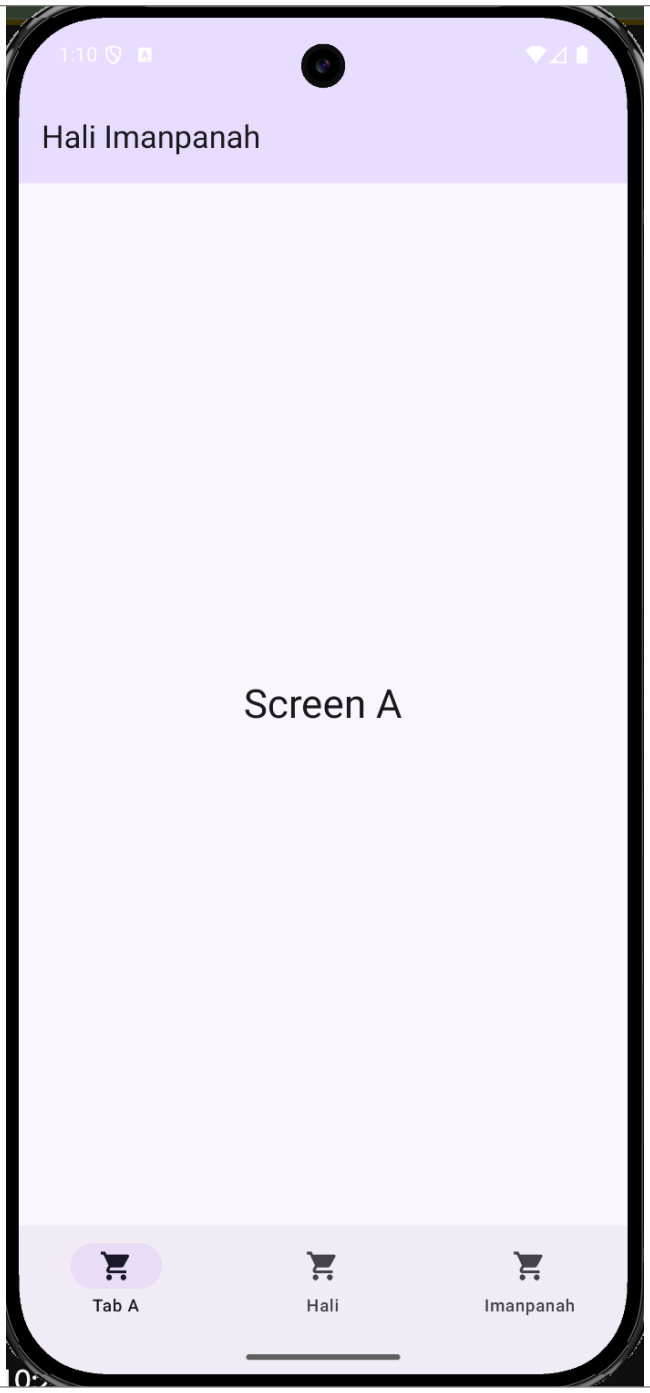
```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun getTopAppBar():Unit {
    return TopAppBar(
        colors = TopAppBarDefaults.topAppBarColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer,
            titleContentColor = MaterialTheme.colorScheme.primary,
        ),
        title = {
            Text("Pat's Top App Bar")
        }
    )
}
```

Then, add this code to the scaffold in MainActivity.kt.

```
topBar = { getTopAppBar() },
```

Exercise 16 (2 marks)

Add an app bar with your name in it on the new screen that you created in Example 9. Show a screenshot of the new screen in the emulator with your app bar in it.


1:10
Hali Imanpanah
Screen A
Tab A
Hali
Imanpanah